

The Model-View-Controller Architecture

Marc Bousse, Mickaël Foursov

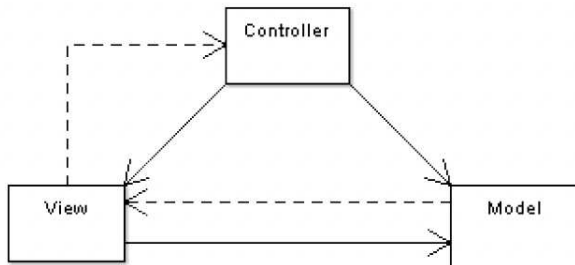
ISTIC/Université de Rennes

22 janvier 2024

- 1 MVC in General
 - Essentials
- 2 MVC before Java 8
 - Collaborations and Responsibilities
- 3 MVC in Java 8
 - From Notification to Binding
- 4 History
 - Origins
 - Platforms & Architectures
 - Architectures

MVC = Model-View-Controller

- MVC is an architecture, not a language nor a library.
- Used to design robust interactive applications
- Considers three parts in any application :
 - Model : handles application logic (interface-independent)
 - View : makes the Model part visible to the user
 - Controller : handles user input, requests updates on the Model and the View parts



How It Works

- 1 A reactive interface for Model is presented to the user by View.
- 2 The user interacts with this interface ; user gestures are handled by Controller
- 3 Controller translates user gestures into user commands
- 4 Controller issues requests on Model and/or View accordingly
- 5 Model changes state if necessary, informing View if it does.
- 6 View refreshes its display, using information from Model
- 7 The user produces new actions, and so on.

An Example : Mail Reader

- Model data : mail folders, messages, titles, texts, etc.
- Model "getter" methods : folder content, message content,...
- Model "setter" methods : read mail, delete message, move message to folder, etc.
- View = mail window with menus, folder list, message list, message text, selections, etc.
- Actions handled by Controller : menu command selection, folder selection, message selection
- Controller calls "getter" + "setter" methods to change folders and messages
- Controller changes selections within View
- Model signals changes to View(s) : changed message list, etc.
- View updates content using "getter" methods.

MVC before Java 8 : the Model Component

- One single Model object encapsulating many collaborators
- View- and Controller-independent
- Model state exposed mainly through *Java Beans* :
 - `private SomeType stateVariable`
 - `public SomeType getStateVariable()`
 - `public void setStateVariable(SomeType state)`
- Complex actions provided by specialized methods :
 - `public void doSomething(...)`
 - `public String somethingStatus()`
- NO direct interaction with user
- NO messages to View/Controller, EXCEPT notifications using Observed/Observer pattern

A Typical Situation : Error & Warning Messages

- How can Model inform user of errors/warnings ?
- Model is UI-independent : calls to View or Controller methods are forbidden !
- Model provides this interface instead :
 - 1 before command execution : boolean precondition methods
 - 2 command methods
 - 3 after command execution : getter methods with status and errors/warnings
- Controller has the initiative :
 - 1 command called on Model only if its preconditions are met
 - 2 after command execution, Controller calls error/warning getter methods
 - 3 error/warning are displayed in View at Controller's initiative

Observer/Observable Design Pattern

- Design pattern : general solution to a recurrent problem
- Language-independent : describes a solution based on collaborating objects
- Observer/Observable : an Observable object informs one or more Observers it has changed
 - an Observable maintains a list of known Observers, which it notifies of its changes
 - an Observer can react to notifications sent by observed Observables
- Besides being "Observable", Model has no explicit knowledge of its observers : Views and Controllers
- Java support : `java.util.Observable` abstract class and `java.util.Observer` interface

MVC before Java 8 : View-Model Relation

- Each Model has one or many Views
- Views are Model-dependent (create a visible image of the Model) :
 - direct reference to Model object (argument of View constructor)
 - View design depends on Model functionality and interface
 - at least one specific View class for each (Model, Interface) pair
- View = built from “off-the-shelf” components (AWT+Swing) :
`javax.swing.JFrame`, `javax.swing.JButton`,...

View-Controller Relation

- Each View has one Controller
- Not really Controller-dependent :
 - Controller is created by the View constructor
 - Controller class depends on View class
 - Controller is Listener for events created by View components

The Controller Component

- Responsibilities :
 - translates user gestures to commands
 - commands local to the interface need View collaboration.
Example : item selection.
 - other commands are delegated to Model. Example : "delete" command.
- Controller is Model-dependent and View-dependent

What's new in Java 8

Before 1.8, Java only had :

- primitive types
- classes
- interfaces
- instances of classes and interfaces : objects

Now it *also* has :

- **functional interfaces**
- instances of functional interfaces :
objects **and lambda expressions**

Functional Interface

Functional Interface : any ordinary Java interface having exactly *one* method definition. For example :

```
public interface Comparator
{ public abstract boolean bigger(String x, String y) ; }
```

Instances are either “plain old Java objects” :

```
Comparator compareObject
    = new ComparatorClass();
```

... or “function values” :

```
Comparator compareFunction =
    (x,y)->{ return x.charAt(0) > y.charAt(0); } ;
```

Using functions as arguments : Lambda Expressions

We can parameterize with *functions* instead of *objects* :

```
// Sort method with array and Comparator arguments:  
public static void mySort(String[] array, Comparator comp)  
{ ... if(comp.bigger(array[i], array[j])) {...}}
```

```
// 1st call taking instance of separate class as argument  
String[] myArray = {"def", "abc"} ;  
Comparator compareObject = new ComparatorClass();  
mySort(myArray, compareObject) ;
```

```
// 2d equivalent call taking lambda expression as argument  
Comparator compareFunction =  
    (x,y)->{ return x.charAt(0) > y.charAt(0); } ;  
mySort(myArray, compareFunction) ;
```

Observer pattern revisited : Java 8 observers

Under Java 8 any observer must implement one of two possible functional interfaces :

```
package javafx.beans;  
public interface InvalidationListener  
{ void invalidated(Observable observable); }
```

```
package javafx.beans.value;  
public interface ChangeListener<T>  
{ void changed(ObservableValue<T> obs, T oldV, T newV); }
```

Which interface choose?

- *InvalidationListener* uses *lazy* evaluation
- ***ChangeListener*** uses *eager* evaluation (preferred!)

Observable and its observers

An *Observable* takes lambda (function) expressions as observers :

```
public interface Observable
{
    void addListener(InvalidationListener l);
    void removeListener(InvalidationListener l);
}
```

In addition, *ObservableValue* has a read-only value of type T :

```
public interface ObservableValue<T>
{
    public void addListener(ChangeListener<T> l);
    public void removeListener(ChangeListener<T> l);
    public T getValue();
}
```


Property

Property<T> extends ObservableValue<T> with :

- void setValue(T)
gives write access to *value*
- void bind(ObservableValue<T> obsValue)
makes *this* an observer to *obsValue*
- bindBidirectional(Property<T> other)
makes *this* an observer to *other*, and vice-versa
- if *a* is bound to *b*, any change to *a*.getValue() triggers
a call to *b*.setValue()
- bind() virtually does this :
Property<T> a, b;
a.addListener((o, oldV, newV)->{b.setValue(newV);});

Implementations of Property

There is a concrete implementation for each basic value type :

- `SimpleIntegerProperty` implements `IntegerProperty`
- `SimpleDoubleProperty` implements `DoubleProperty`
- `SimpleStringProperty` (etc.)
- `SimpleListProperty<E>` (also implements `List<E>`)
- `SimpleMapProperty<E>` (also implements `Map<E>`)
- ...and more in `javafx.beans.property`

JavaFX Bean

When using JavaFX, follow *Java/JavaFX Bean* conventions :

```
private StringProperty myValue
    = new SimpleStringProperty("foo");

public StringProperty myValueProperty()
{ return myValue; }

public String getMyValue()
{ return myValueProperty().get(); }
    // get() works like getValue()

public void setMyValue(String myValue)
{ myValueProperty().set(myValue); }
    // set() works like setValue()
```

Exposing a read-only property

- Problem : `public StringProperty myValueProperty()` gives write access
- instead, use ***ReadOnlyStringWrapper*** :

```
private ReadOnlyStringWrapper myValue ;  
private void setMyValue(String s) { myValue.set(s); }
```

```
public ReadOnlyStringProperty myValueProperty()  
{ return myValue.getReadOnlyProperty(); }
```

```
public String getMyValue()  
{ return myValueProperty().get(); }
```

```
// Not allowed in this context:  
// myValueProperty().set("abc")
```

Origins of MVC

- Early object-oriented programming language : Smalltalk
- Invented at Xerox PARC (Palo Alto Research Centre) : Smalltalk-78, Smalltalk-80
- Integrated graphical programming environment
- First environment with modern GUI : browsers, scrollable lists, mouse, cut/paste,...
- GUI composed of general purpose objects : Views & Controllers
- First described by Trygve Reenskaug in 1979
- Integrated in Smalltalk class library, for reuse in graphical UI developments.

Late Recognition

- Only used by researchers at first
- Arrival of Java + Web in the late nineties : need for big UI libraries
- Many frameworks are MVC-based : WinForms (.NET), Struts (Java), Swing (Java), J2EE (Java), Catalyst(Perl)
- Strict adherence to MVC is up to the programmer...

MVC on Various Platforms

- Java SWING : see above
- Java Platform, Enterprise Edition (J2EE)
 - Model = entity bean (persistent JavaBean)
 - View = Java Server Page
 - Controller = servlet
- other platforms have additional MVC frameworks : for example *Catalyst* for Perl.

Alternative Architectures

- Model 1 : used for Java Web applications. No separation between presentation logic (view) and business logic (model)
- Model 2 : improvement on Model 1, see J2EE above
- PAC (Joëlle Coutaz, 1987) = Presentation, Abstraction, Control
 - UI structured as a hierarchy of *agents*
 - each agent is a combination of P, A, and C
 - only Control is visible to other agents